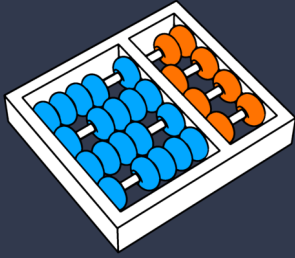




UNICAMP



# Introdução ao LangGraph

Prof. Dr. Julio Cesar dos Reis  
dosreis@unicamp.br

# Agenda

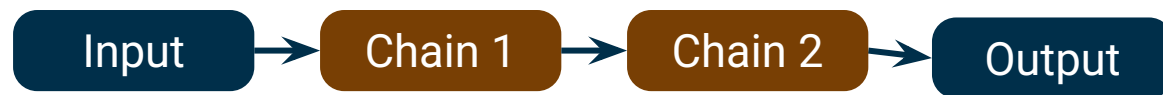
- ❑ LangChain vs. LangGraph
- ❑ Agente como grafo
- ❑ Estados
- ❑ Nós
- ❑ Transições
- ❑ Roteamento condicional
- ❑ Ciclos e controle de execução



# LangChain vs LangGraph



# LangChain



---

## Aspecto



### Modelo

Cadeias e componentes

**Unidade principal** *Chain, Runnable, Tool, Agent*

**Fluxo** Geralmente sequencial ou composto

**Estado** Pode existir, mas não é o centro da abstração

**Condições** Possíveis, mas menos naturais em fluxos complexo

**Ciclos** Normalmente aparecem dentro de agentes prontos

# ‘Limitações’ com LangChain para agentes

- ❑ ✗ Sem loops (não replaneja)
- ❑ ✗ Controle de fluxo rígido
- ❑ ✗ Difícil gerenciar estado complexo
- ❑ ✗ Difícil efetuar planejamento iterativo
- ❑ ✗ Não suporta bem decisões condicionais

# ‘Limitações’ com LangChain para agentes

- ❑ **Como fazer um agente tentar novamente se errar?**

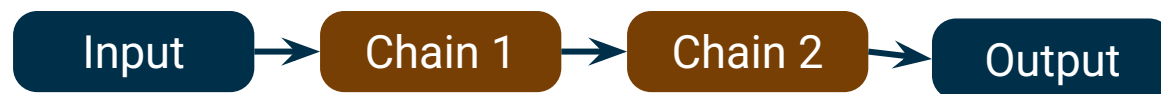
- ❑ Agentes precisam

- ❑ Iterar
- ❑ Tomar decisões
- ❑ Usar ferramentas
- ❑ Manter memória

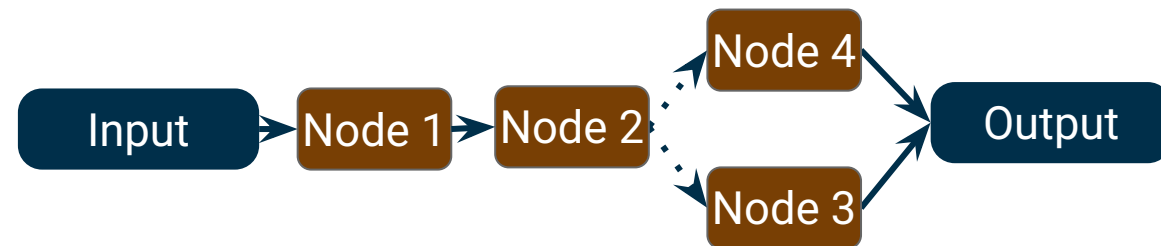
# LangGraph



LangChain



LangGraph



Aspecto

LangGraph

**Modelo**

Grafos de execução

**Unidade principal**

Estado, nó e aresta

**Fluxo**

Explícito, ramificado e controlado (dinâmico)

**Estado**

Estado é elemento central (compartilhado)

**Condições**

Roteamento condicional é parte do desenho

**Ciclos**

Ciclos podem ser modelados explicitamente

# LangGraph vs. LangChain

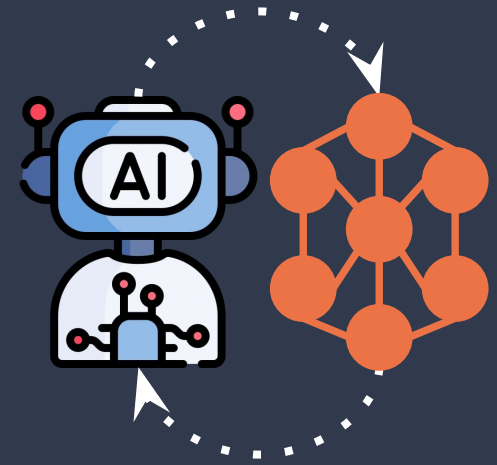
| Aspecto    |  LangChain |  LangGraph |
|------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Modelo     | Pipeline                                                                                     | Grafo                                                                                         |
| Fluxo      | Linear                                                                                       | Dinâmico                                                                                      |
| Loops      | Difícil                                                                                      | Nativo                                                                                        |
| Estado     | Limitado                                                                                     | Central                                                                                       |
| Controle   | Implícito                                                                                    | Explícito                                                                                     |
| Uso típico | RAG simples                                                                                  | Agentes                                                                                       |



# LangGraph vs. LangChain

- ❑ O LangGraph não substitui o LangChain.
- ❑ LangChain continua sendo relevante para construir aplicações com LLMs
  - ❑ Especialmente na integração com modelos, prompts, ferramentas, parsers e retrievers.

# Agente como Grafo



# Agente como Grafo

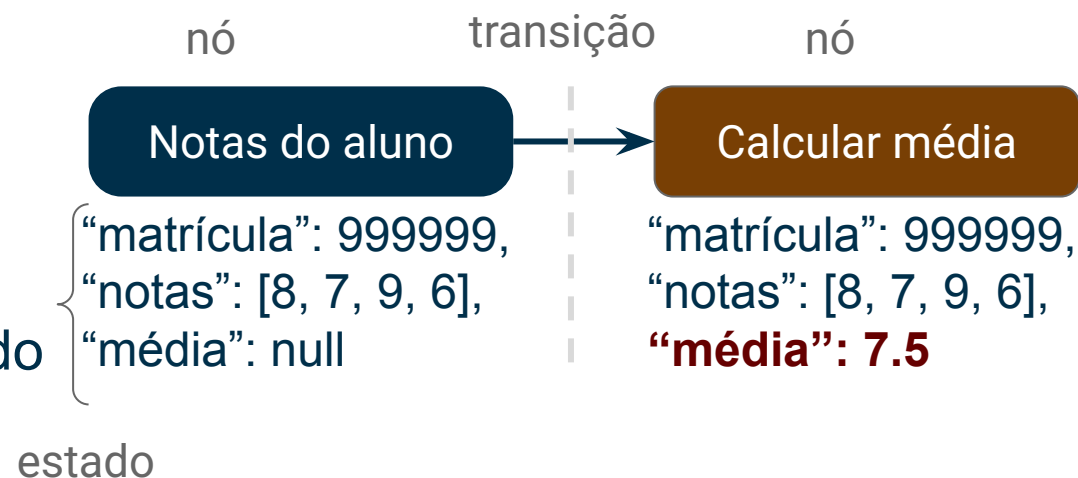
- ❑ Um agente pode ser entendido como um sistema que recebe uma entrada
- ❑ Mantém informações ao longo da execução e decide qual operação realizar em seguida.

# Grafo de Execução

- ❑ LangGraph representa o funcionamento de um agente por meio de um **grafo de execução**.

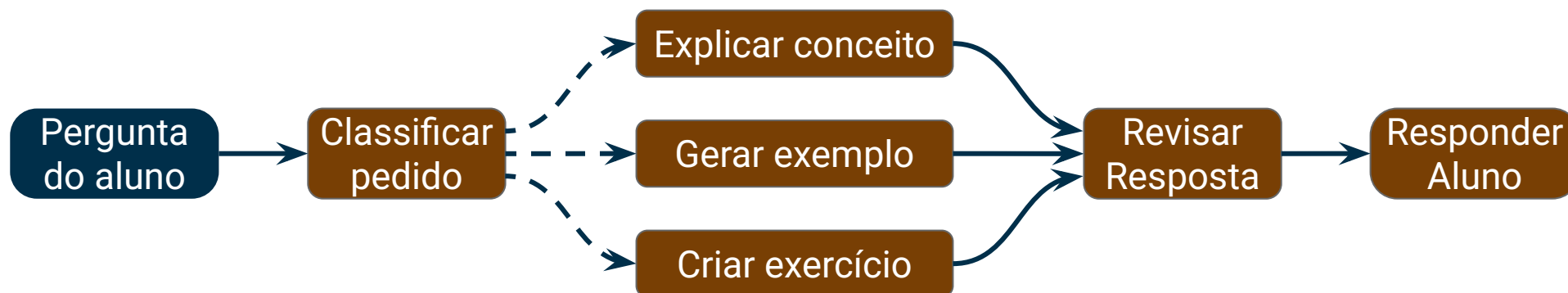
## Elementos do grafo

- Estado** → Guarda as informações da execução
- Transições** → Definem o próximo passo
- Nós** → Executam tarefas e atualizam o estado



# Grafo de Execução

- ❑ O agente é modelado como um grafo porque sua execução pode seguir caminhos diferentes dependendo do **estado atual**.



**Estados**

# Estado (*State*) em LangGraph



## "Memória" do Agente

Funciona como a **memória compartilhada** que orquestra a inteligência do agente.

### Persistência e Evolução

- Informações que evoluem ao longo da execução do grafo.

### Conhecimento Instantâneo

- Representa exatamente o que o agente "sabe" em cada momento.



```
// Exemplo de Estrutura de Estado

state = {
    "input": "Qual é a capital da
França?",
    "messages": [],
    "steps": 0
}
```

# Estado como contrato

- ❑ O estado estabelece um contrato entre todos os nós do grafo.
  - ❑ Estrutura compartilhada entre os nós
- ❑ Qualquer informação que precise ser passada de um nó para outro deve estar representada no estado



# Imutabilidade relativa

- ❑ Cada nó não substitui o estado inteiro
  - ❑ Apenas declara o que mudou.
  - ❑ LangGraph aplica essas atualizações de forma controlada, mantendo a consistência entre os passos.

# Estado vs. Variáveis locais

- ❑ Estado é diferente de variáveis locais de uma função
- ❑ O estado é visível por todos os nós
- ❑ O estado sobrevive a cada transição
- ❑ Permite que decisões tomadas em um nó influenciam o comportamento de nós posteriores

**Nós**

# Nós (*Nodes*)

- Unidades de trabalho do grafo -> Cada nó = “uma operação cognitiva”
- Cada nó representa uma responsabilidade bem definida.

## Exemplos de Uso:

-  Classificar uma intenção
-  Consultar base de conhecimento
-  Gerar uma resposta
-  Tomar uma decisão

## ESTRUTURA TÍPICA

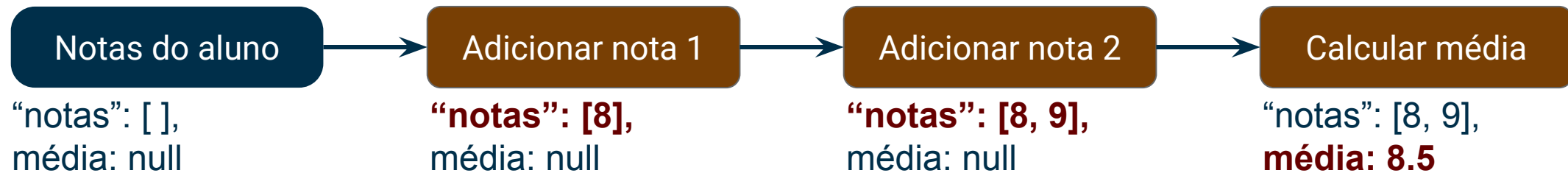
```
def node(state):  
    # processa a lógica  
    return state
```

# Boas práticas

- ❑ Um nó **bem projetado** tem um escopo bem definido.
  - ❑ uma função bem definida
- ❑ Essa separação facilita o teste, a reutilização e a compreensão do comportamento do agente como um todo.

# Nó como transformador de estado

1. Um nó lê o estado atual
2. Executa sua lógica
  - que pode envolver uma LLM, uma ferramenta ou uma regra de negócio
3. Devolve apenas as partes do estado que foram modificadas



# Lógica dos Nós

- ❑ Nós podem conter qualquer tipo de lógica
- ❑ Internamente, um Nó pode fazer variados tipos de operações
  - ❑ Invocar um modelo de linguagem
  - ❑ Consultar uma API
  - ❑ Aplicar uma regra determinística
  - ❑ Combinar várias dessas estratégias.

# Transições

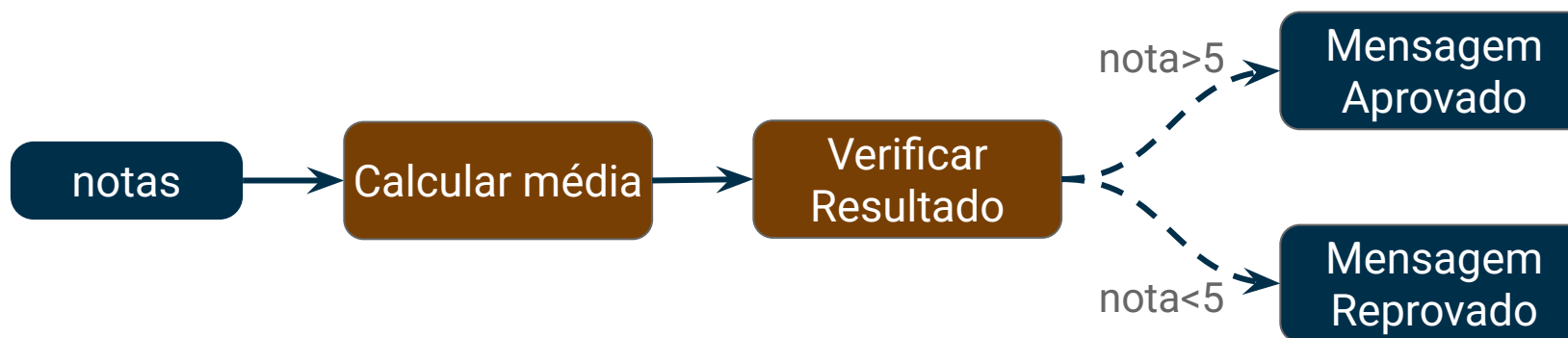


# Transições (ou arestas - *edges*)

- ❑ Definem a sequência de execução
  - ❑ Após um nó terminar, qual será o próximo?
- ❑ A estrutura que transforma nós isolados em um fluxo coerente de raciocínio.

# Tipos de transições

- ❑ **Simple**s conectam dois nós de forma direta e incondicional.
  - ❑ o fluxo sempre segue aquele caminho.
- ❑ **Condicionais** delegam a decisão para uma função que analisa o estado atual.

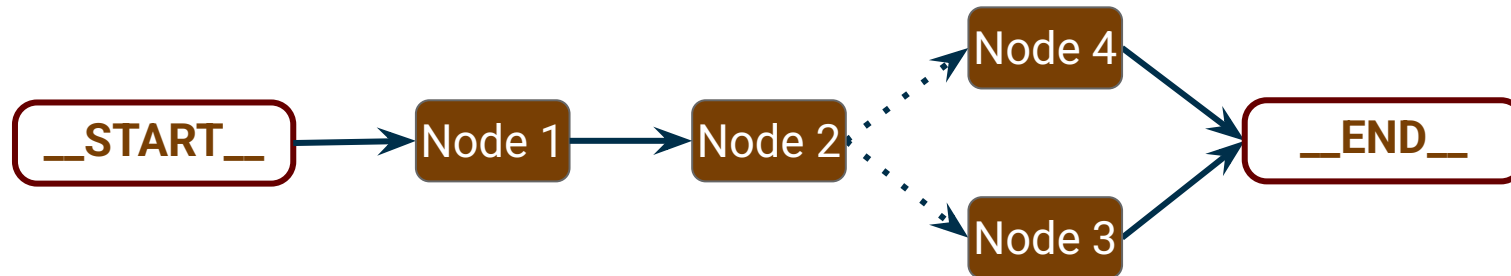


# Transições definem a intenção do designer

- ❑ Transições codificam o raciocínio esperado do agente
  - ❑ Quais caminhos são possíveis?
  - ❑ Quais são obrigatórios?
  - ❑ Em que condições cada um se aplica?

# Ponto de entrada e Ponto de saída

- ❑ Todo grafo possui um ponto de entrada explícito (**START**)
- ❑ Um ou mais pontos de término (**END**).
- ❑ Estrutural que torna o comportamento do agente previsível e auditável.



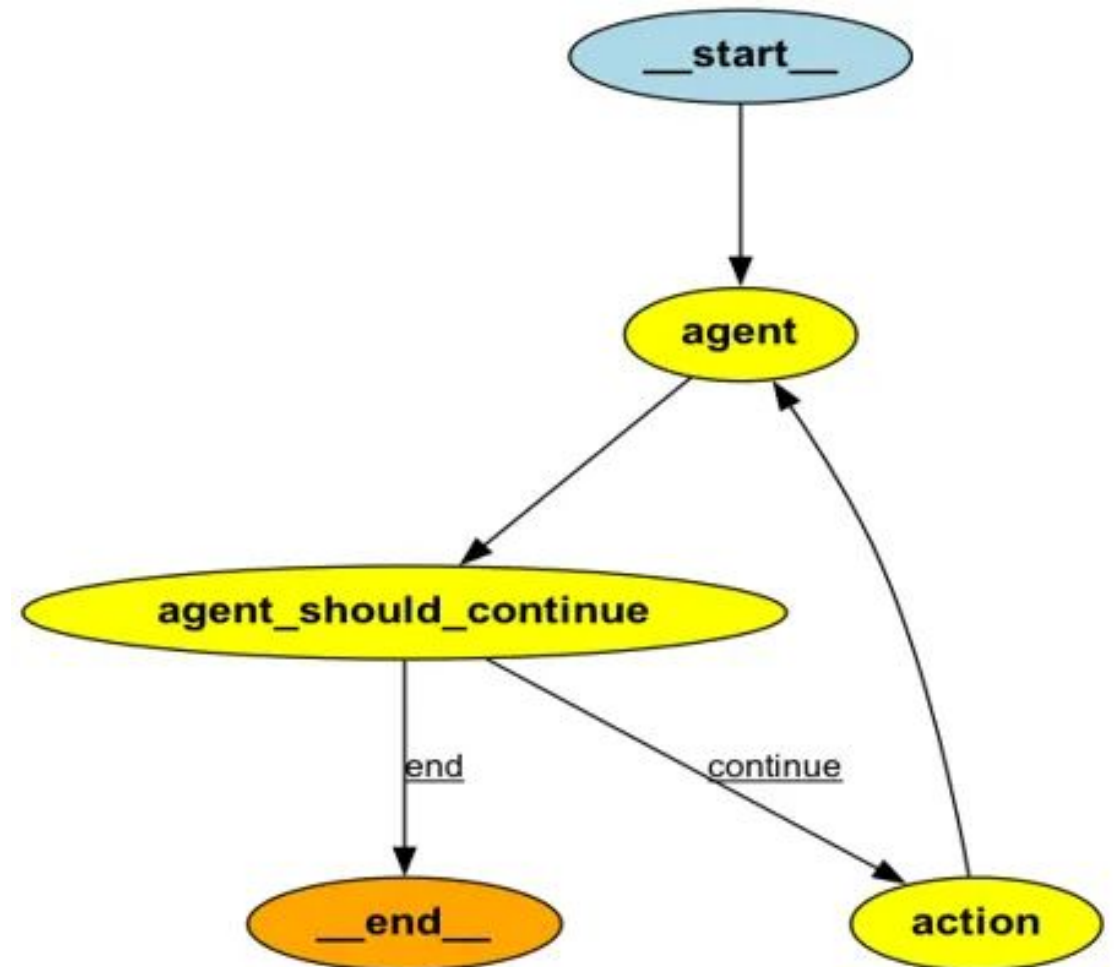
# Ciclos e Controle de Execução

# Ciclos são necessários

- ❑ Agentes inteligentes raramente resolvem problemas em linha reta.
- ❑ Necessitam **revisar**, **refinar** e **corrigir**.
- ❑ Ciclos permitem que o grafo expresse esse comportamento iterativo de forma natural e controlada
- ❑ Permite:
  - ❑ reflexão
  - ❑ replanejamento
  - ❑ retry

# Grafo Cíclico

- ❑ Um grafo cíclico permite a criação de loops em sua estrutura
- ❑ Possibilita a execução repetida de determinados nós ou sequências de nós
- ❑ Crucial para tarefas que exigem processos iterativos ou a revisitação de decisões em aplicações de LLM.



# Ciclo como mecanismo de reflexão

- ❑ Um agente pode:
  - a. gerar uma resposta
  - b. avaliá-la com um segundo nó e, se a qualidade não for satisfatória,
  - c. voltar ao passo de geração com informações adicionais.
- ❑ Esse padrão é chamado de “reflexão”.



# Controle de Execução

- ❑ LangGraph oferece mecanismos para evitar laços infinitos
  - ❑ Como limites de recursão configuráveis
- ❑ É possível pausar a execução aguardando *input* humano (padrão *Human-in-the-Loop*).

# Síntese

- ❑ LangGraph instrumentaliza agentes como grafos de execução explícitos.
- ❑ O estado é o contrato compartilhado entre os nós.
- ❑ Nós modificam o estado e há transição entre nós.
- ❑ Roteamento condicional permite ramificação baseada no estado.

# Referências

- ❏ [Documentação oficial do LangGraph](#)
- ❏ [LangGraph – Graph API:](#)
- ❏ [Repositório oficial do LangGraph](#)